Speed is Confidence

Anonymous Authors[1]

Abstract

Biological neural systems must be fast but are energy-constrained. Evolution's solution: act on the first signal. Winner-take-all circuits and time-to-first-spike coding implicitly treat *when* a neuron fires as an expression of confidence.

We apply this principle to Tiny Recursive Models (TRM) (**?**). On Sudoku-Extreme, a baseline TRM achieves 85.5% $\pm$ 1.3%. But a key diagnostic reveals untapped potential: 89% of failures are *selection* problems—the model can solve these puzzles with a different random initialization. The true capability ceiling is 99%, not 86%.

Halt-first ensembling unlocks this potential: by selecting the first model to halt, we achieve 97% accuracy vs. 91% for probability averaging—while requiring 10$\times$ fewer reasoning steps. But can we internalize this as a training-only cost? Yes: by maintaining K=4 parallel latent states and backpropping only through the lowest-loss "winner," we achieve **96.9% $\pm$ 0.6%** accuracy—matching ensemble performance at 1$\times$ inference cost.

As in nature, this work was also resource constrained: all experiments used a single RTX 5090. A modified SwiGLU (**?**) made Muon (**?**) and high LR viable, enabling baseline training in 48 minutes and full WTA (K=4) in 6 hours—compared to TRM's 20 hours on an L40S.

Code: https://anonymous.4open.science/r/sic-B854

1. Introduction

When faced with uncertainty, biological neural systems don't perseverate indefinitely—they act on the first confident signal. This principle, refined by millions of years of evolution, underlies rapid decision-making from prey detection to social judgment. In cortical winner-take-all circuits, competing neural populations race to threshold; the first to fire suppresses alternatives and triggers action (**?**). The speed of convergence itself carries information: a fast response indicates a clear, unambiguous stimulus.

We apply this biological insight to improve neural network ensembles. Standard ensemble methods average predictions across models, treating all outputs equally regardless of when each model reaches its answer. For iterative reasoners with adaptive computation time (ACT) (**?**), this discards valuable information: models that converge quickly have typically found "clean" solution paths, while those still deliberating are often stuck or uncertain.

Our key observation is that **inference speed is an implicit confidence signal**. When multiple models reason in parallel, the first to halt is most likely correct. This simple selection rule—which we call *halt-first ensembling*—improves accuracy by 5.7 percentage points over probability averaging while requiring 10$\times$ fewer reasoning steps on Sudoku-Extreme.

This raises a natural question: can we internalize this benefit within a single model? Training and deploying multiple models is expensive. We show that the diversity exploited by halt-first selection comes primarily from different random initializations during training. By maintaining K parallel latent initializations within one model and training with a winner-take-all (WTA) objective, we achieve ensemble-level accuracy with single-model deployment cost.

Contributions.

1. We demonstrate that halt-first selection outperforms probability averaging for ensembles of iterative reasoners, achieving 97.2% accuracy vs. 91.5% while requiring 10$\times$ fewer reasoning steps.
2. We introduce *oracle-first training*, a WTA method that internalizes ensemble diversity within a single model, achieving 96.9% $\pm$ 0.6% accuracy with zero inference overhead.
3. We show that 89% of baseline failures are *selection* problems, not capability limits—the model can solve these puzzles with different initializations. This reveals that the accuracy ceiling is 99%, not 86%.
4. We discover that Muon optimization fails with standard

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

SwiGLU and provide a fix (RMSNorm before down-projection), enabling training in 48 minutes on consumer hardware (RTX 5090) vs. hours on datacenter GPUs for baseline TRM—making rapid experimentation feasible.
5. We conduct extensive ablations identifying critical design choices: carry policy, SVD-aligned initialization, and z_L diversity.

2. Background

2.1. Tiny Recursive Models

Tiny Recursive Models (TRM) (**?**) achieve strong performance on constraint satisfaction problems using a remarkably simple architecture: a small neural network that iteratively refines its predictions through recursive application: The model maintains two latent states: z_L for "low-level"

Algorithm 1 TRM with ACT (Inference)

for $t = 0, \ldots, n_{\text{ACT}}-1$ # n_{ACT}=16
  for $i = 0, \ldots, n_{\text{H}}-1$ # n_H=6
    $z_L, z_H \leftarrow \text{stop}(z_L), \text{stop}(z_H)$
    for $j = 0, \ldots, n_{\text{L}}-1$ # n_L=9
      $z_L \leftarrow f(\text{embed}(x;\theta) + z_L + z_H;\theta)$
    end
    $z_H \leftarrow f(z_L + z_H;\theta)$
  end
  $y, q \leftarrow h_{\text{puzzle}}(z_H;\theta), h_{\text{halt}}(z_H;\theta)$
  if $q > 0$ **then**
    break
  end
end

details (updated n_{L} times per H-cycle, i.e., one outer loop iteration) and z_H for "high-level" reasoning (updated once per H-cycle). The $\text{stop}(\cdot)$ operator detaches its argument from the computation graph, preventing gradient flow between H-cycles and enabling stable training.

On Sudoku-Extreme (17-clue puzzles, the minimum for uniqueness), a 7M parameter TRM achieves $\sim$86% puzzle accuracy—substantially outperforming larger language models while using a fraction of the parameters.

2.2. Adaptive Computation Time

Rather than fixing the number of iterations T, TRM can learn *when to stop* via Adaptive Computation Time (ACT) (**?**). The model outputs a halting probability $q_{\text{halt}} \stackrel{\text{def}}{=} \text{sigmoid}(q)$ at each step, trained to predict whether the current output is correct. When $q_{\text{halt}} > 0.5$, the model "commits" to its answer.

This creates an implicit tradeoff: easy puzzles should halt quickly (high confidence after few iterations), while hard puzzles may require more computation. In practice, a well-trained TRM solves most Sudoku puzzles in 1–3 steps, with a long tail requiring up to 16 iterations.

3. Method

3.1. The Ensemble Puzzle

Standard practice for improving accuracy is ensembling (**?**): train multiple models with different seeds and average their predictions. For TRM on Sudoku-Extreme:

Method	Puzzle Acc.	ACT Steps
Single model (avg)	85.0%	16
Ensemble (3 seeds)	88.52%	48
+ TTA (4 rotations)	91.51%	192

Ensemble plus test-time augmentation yields +6.5% accuracy, but at 12$\times$ the compute cost. This presents a puzzle: ensembling helps, but the standard approach of probability averaging seems wasteful. Each model in the ensemble has its own "opinion" about when to stop reasoning, yet we ignore this timing information entirely. Can we do better by paying attention to *when* models commit to their answers?

We begin with a biological intuition and follow it to its logical conclusion.

Biological neural systems face a fundamental constraint: metabolic energy is scarce, yet decisions must be fast. Evolution's solution is to *act on the first confident signal*. In cortical winner-take-all circuits (**?**), competing neural populations race to threshold, with the first to fire suppressing alternatives. Spiking neural networks formalize this as time-to-first-spike (TTFS) coding (**??**), where information is encoded in *when* neurons fire rather than *how often*—achieving both speed and energy efficiency.

This principle suggests a strategy for ensembles of iterative reasoners: instead of averaging probabilities, **select the first model to halt**.

3.2. Halt-First Ensemble

The procedure is simple: run all ensemble members in parallel, each performing its own ACT iterations. When any model's halting signal exceeds threshold, that model provides the final answer. The results are striking:

Method	Acc.	Steps	Speedup
Prob. avg. (12)	91.5%	192	1$\times$
Halt-first (12)	**97.2%**	**18.5**	**10**$\times$

Halt-first selection improves accuracy by 5.7 percentage points while requiring 10$\times$ fewer reasoning steps. This validates the biological intuition: **inference speed is an**

implicit confidence signal. When a reasoning process converges quickly, it indicates the model has found a "clean" solution path—one without contradictions or backtracking. Probability averaging destroys this signal; halt-first selection preserves the information that evolution discovered: the reasoner that finishes first is most likely correct. We analyze the halting dynamics in detail in **??**.

3.3. Oracle-First Training

The halt-first result raises a natural question: can we internalize this benefit within a single model? Deploying multiple models is expensive—we want the accuracy of an ensemble with the deployment cost of one.

Our key insight is that ensemble diversity comes from different random initializations during training. We can internalize this diversity by maintaining K parallel latent initializations *within* one model and letting them compete. During training, we run $K{=}4$ parallel hypotheses; at inference, we run just one.

Specifically, we maintain K parallel low-level states $\{z_{Lk}\}_{k=1}^{K}$, each from a learned initialization $L_{\text{init},k}$, while sharing the high-level state z_H. At each step, all K heads process the input in parallel, and the **winner-take-all** (WTA) rule selects which receives the gradient:

$$k^* = \arg\min_k \mathcal{L}_{\text{CE}}(y_k, y_{\text{target}}) \tag{1}$$

Only the winning head contributes to the loss. This mirrors halt-first selection: at inference, the fastest model wins; during training, the most accurate head wins. Both reward the same underlying property: having found a clean solution path.

Why does lowest loss correlate with fastest halting? Both are downstream effects of the same cause: *solution path quality*. When iterative refinement encounters no contradictions, z_L stabilizes quickly—the fixed-point iteration has a large basin of attraction. This rapid stabilization produces both correct outputs (low loss) and confident halting signals (the model detects its own convergence). Conversely, when refinement oscillates or drifts, both accuracy and halting confidence suffer. Oracle-first training thus learns to find the same "clean paths" that halt-first selection exploits.

Training dynamics. A key detail: TRM training "unpacks" the outer loop of Algorithm 1. Each gradient step performs one H-cycle, carrying z_H and z_L to the next step. This means the K parallel chains evolve across multiple gradient updates, not within a single forward pass.

Carry policy. After each step, we must decide how to propagate states to the next iteration. We use `copy` for z_H: the winner's z_H is copied to all chains (Algorithm 2, line 10). For z_L, we use `all`: each chain keeps its own z_L. This allows heads to benefit from shared high-level progress while maintaining low-level diversity (see **??** for ablations).

Algorithm 2 Oracle-First Training (one step)

Input: x, y_{target}, carried z_H, $\{z_{Lk}\}_{k=1}^{K}$
for $k = 1$ to K
  $y_k, q_k, z_{Hk}, z_{Lk} \leftarrow \text{TRM}(x, z_H, z_{Lk}; \theta, n_{\text{ACT}} = 1)$
end
$k^* \leftarrow \arg\min_k \mathcal{L}_{\text{CE}}(y_k, y_{\text{target}})$ *# winner*
$c \leftarrow \mathbf{1}[y_{k^*} = y_{\text{target}}]$ *# all correct*
$\mathcal{L} \leftarrow \mathcal{L}_{\text{CE}}(y_{k^*}, y_{\text{target}}) + \lambda \mathcal{L}_{\text{BCE}}(q_{k^*}, c)$
Backprop through $\mathcal{L}$
$z_H \leftarrow z_{Hk^*}$ *# winner's z_H copied to all*

Here $\mathcal{L}_{\text{CE}}$ is cross-entropy with label smoothing ($\alpha{=}0.2$), $\mathcal{L}_{\text{BCE}}$ is binary cross-entropy for the halting signal where q_k is the halting logit and $\text{sigmoid}(q_k)$ is the halting probability, and $\lambda{=}0.05$.

Counter-intuitive design. WTA training deliberately spends compute on reasoning chains that won't contribute to the gradient. This apparent waste enables exploration: losing heads probe alternative paths, and winner selection identifies which succeeded.

3.4. Faster Training

WTA training requires K forward passes per iteration—a $4\times$ increase for $K{=}4$. With only a single GPU, efficient training is essential. We combine two techniques that reduce training time from days to hours, making rapid experimentation possible.

3.4.1. Muon Optimizer

We use Muon (**?**) for 2D weight matrices (lr=0.02, wd=0.005) and AdamW for embeddings, heads, and biases (lr=10^{-4}, wd=1.0, β=(0.9, 0.95)). Muon's orthogonalized momentum enables substantially higher learning rates than AdamW alone, accelerating convergence. However, Muon is magnitude-invariant, so bias-like parameters require AdamW. Learning rate follows a cosine schedule.

Selective regularization. This split-optimizer design creates an asymmetric regularization structure: the core reasoning weights (MLPMixer blocks) receive minimal regularization via Muon's low weight decay, while embeddings and output heads receive strong regularization via AdamW's high weight decay. Combined with label smoothing ($\alpha{=}0.2$), this encourages the core model to learn expressive representations while preventing the input/output interfaces from overfitting. In contrast, baseline TRM applies uniform high weight decay (1.0) to all parameters,

which may over-constrain the reasoning layers.

Modified SwiGLU. We observe that Muon with high LR underperforms with $\mathrm{SwiGLU}(x) \stackrel{\text{def}}{=} \mathrm{SiLU}(g) \odot v$ where $[g, v] = xW$ (split in half). Evidently the issue stems from Muon being proximal descent with spectral norm (Stiefel manifold; orthogonal matrices), but since $\mathrm{SiLU}(g) = \mathrm{sigmoid}(g) \cdot g$, we have $\mathrm{SwiGLU} = \mathrm{sigmoid}(g) \odot g \odot v$, which carries magnitude. To remedy, we introduce an additional normalization: $\mathrm{SwiGLU}_{\text{muon}}(x) = \mathrm{sigmoid}(g) \odot \mathrm{RMSNorm}(g \odot v)$. This bounds the magnitudes and prevents Muon's orthogonalized updates from being dominated by large values.

3.4.2. SVD-Aligned Initialization

The K initialization vectors $L_{\text{init},k}$ determine hypothesis diversity. Naive random initialization risks "dead" heads—initializations that point in directions the network cannot use effectively. We address this by computing the top singular vectors of the first layer's weight matrix and ensuring all K heads have equal alignment with this subspace. This provides equal "effective signal strength" across heads while maintaining diversity in the orthogonal complement.

3.5. Intuition: Why Diverse z_L with Shared z_H

We offer a speculative interpretation of why maintaining K parallel z_L states while sharing z_H works well. This is intuition, not a formal result.

Bilevel fixed-point structure. TRM computes a nested equilibrium:

$$z_L^* = f(z_L^*, z_H, z_x) \quad \text{(inner: local refinement)} \tag{2}$$

$$z_H^* = g(z_H^*, z_L^*) \quad \text{(outer: global integration)} \tag{3}$$

where $z_x = \mathrm{embed}(x)$ is the embedded input (fixed throughout iteration). The inner loop equilibrates z_L for fixed z_H; the outer loop updates z_H given the equilibrated z_L^*. This creates a natural hierarchy: z_L handles cell-level constraint propagation while z_H maintains puzzle-level state.

Schur complement structure (intuition). The implicit gradient through this bilevel system reveals an elegant structure. Writing the joint state $Z = (z_L, z_H)$ and joint Jacobian in block form:

$$I - J = \begin{bmatrix} I - J_L & -K_L \\ -K_H & I - J_H \end{bmatrix} \tag{4}$$

where J_L, J_H are self-Jacobians and $K_L = \partial f / \partial z_H$, $K_H = \partial g / \partial z_L$ capture cross-coupling. Block inversion yields a Schur complement:

$$S = (I - J_H) - K_H (I - J_L)^{-1} K_L \tag{5}$$

The effective outer inverse S^{-1} equals the naive inverse $(I - J_H)^{-1}$ *corrected* by how inner-outer coupling (K_H, K_L) interacts through the inner equilibrium $(I - J_L)^{-1}$. This creates nested Neumann series: the inner equilibrium "solves" for z_L^* given z_H, then the outer loop optimizes the simpler problem of finding good z_H—divide and conquer. In practice, TRM truncates these series, but the bilevel structure may make truncation less harmful: deep effective computation with shallow gradient paths.

Architectural asymmetry stabilizes the inner loop. L-cycles receive $z_L + z_H + z_x$ (three unit-variance terms), while H-cycles receive only $z_H + z_L$ (two terms). Assuming approximate independence, z_L contributes $\frac{1}{3}$ of L-cycle input variance versus $\frac{1}{2}$ for H-cycles. The Jacobian $J_L = \partial f / \partial z_L$ is thus smaller: the output is "anchored" by context $(z_H + z_x)$ and less sensitive to z_L. The fixed input z_x stabilizes L-cycles in a way H-cycles lack.

Why diverse z_L. Different z_L initializations explore different basins of attraction in the inner equilibrium landscape. With K parallel initializations, we sample K potential solution paths. The well-conditioned inner loop means these paths can diverge meaningfully without destabilizing training.

Why shared z_H. The `copy` policy copies the winner's z_H to all chains. This propagates successful high-level state (puzzle-level progress) while preserving low-level diversity. It also decorrelates each chain's z_H from its own z_L, strengthening the independence assumption in the variance argument above.

4. Related Work

Recursive reasoning models. TRM (**?**) shows that small networks with iterative refinement can outperform larger models on constraint satisfaction. We extend TRM with multi-head stochastic search, recovering ensemble diversity through parallel latent initializations. Deep Improvement Supervision (**?**) provides explicit intermediate targets; this complements our implicit winner-take-all dynamics.

Adaptive computation. ACT (**?**) learns when to stop iterating; PonderNet (**?**) reformulates halting with unbiased gradients; Universal Transformers (**?**) apply depth-adaptive computation to transformers. Our contribution: the halting signal encodes confidence, making first-to-halt selection superior to probability averaging.

Early-exit and dynamic ensembles. Cascaded ensembles (**?**) run members *sequentially*, exiting when confidence exceeds a threshold. Dynamic Ensemble Selection (DES) (**?**) chooses models via k-NN competence estimation.

Our halt-first selection differs fundamentally: we run models *in parallel* and use timing itself as the confidence signal. This requires no explicit confidence scoring, and achieves both higher accuracy and lower latency than sequential approaches.

Portfolio solvers. Algorithm portfolios (**??**) run multiple solvers in parallel, terminating when any succeeds—optimal for Las Vegas algorithms with unpredictable runtimes. We apply this "first to finish" principle to neural iterative reasoners, with ACT halting as the termination signal.

Diverse solution strategies. PolyNet (**?**) learns multiple complementary construction policies for combinatorial optimization, selecting via explicit scoring. We share the goal of implicit diversity but differ in mechanism: we maintain diverse *latent initializations* competing via WTA, selecting by oracle (training) or halting time (inference) rather than learned scores.

Winner-take-all and competitive learning. WTA appears in competitive learning (**?**), mixture of experts (**?**), and sparse attention (**?**). We apply WTA to *latent initializations*, creating competition among reasoning trajectories rather than network components or experts.

Test-time compute scaling. Chain-of-thought, tree search, and iterative refinement trade inference compute for accuracy. Rather than uniformly allocating compute across all inputs, halt-first selection achieves *better* accuracy with *less* compute by letting the model adaptively allocate based on difficulty—easy inputs halt immediately, hard ones iterate longer. WTA training internalizes this adaptive allocation within a single model.

Biological inspiration. Time-to-first-spike coding (**??**) encodes information in *when* neurons fire. Cortical WTA circuits (**?**) suppress alternatives at threshold. Thousand Brains theory (**?**) proposes parallel cortical models racing to consensus. Human decision-making shows analogous speed-accuracy tradeoffs (**??**). Our halt-first ensemble embodies these principles: parallel reasoners race, first to halt wins. Our findings suggest neural networks exhibit similar dynamics—confident solutions emerge faster during iterative refinement.

5. Experiments

5.1. Setup

We use Sudoku-Extreme: 1,000 training puzzles (with 8× dihedral augmentation and digit permutation) and a randomly sampled 38,400 puzzle subset of 422,786 test puzzles; all have exactly 17 given digits (the minimum for uniqueness). All experiments use a single NVIDIA RTX 5090 (32GiB VRAM), with training completing in 48 minutes for baseline (K=1, 8k steps) to 6 hours for WTA (K=4, 36k steps). The architecture is a 2-layer MLP-Mixer (**?**), 512 hidden (7M params), no attention, with n_{H}=6, n_{L}=9 (increased from 3, 6 in original TRM; this improves our baseline to 85.5% ± 1.3% from their ∼80%) and modified SwiGLU to accommodate Muon. We report puzzle accuracy (fraction solved completely) and cell accuracy (fraction of cells correct), using the halting signal with a maximum of 16 reasoning steps.

Method	Cell	Puzzle	Cost	Eff.‡
Baseline (K=1)	94.5%	85.5±1.3%	1×	85.5
Inference-time methods				
TTA-8 (test-time aug.)	n/a	97.3%	8×	12.2
Halt-first (12 ch.)	n/a	97.2%	1.5×†	64.8
WTA training (ours, K=4)				
Fresh init	**98.7%**	**96.9±0.6%**	**1×**	**96.9**
Fixed init	98.7%	96.9±0.7%	1×	96.9

Table 1. Results on Sudoku-Extreme. Cost is inference cost relative to baseline. WTA achieves ∼**97%** accuracy at 1× cost—Pareto-dominating all other methods. Fresh and fixed init achieve equivalent accuracy (96.9%); fresh has lower variance (±0.6% vs ±0.7%). †Effective cost after early halting (86% halt at step 0). ‡Efficiency = Puzzle Acc / Cost.

5.2. Main Results

?? shows our main results. Key findings:

1. **Selection is the bottleneck.** A TTA diagnostic reveals 89% of baseline failures are selection problems (the model *can* solve the puzzle with a different digit permutation). The ceiling is 99.4%, not 85.5%.
2. **WTA training internalizes diversity.** K=4 z_L heads achieve 96.9% ± 0.6% single-pass accuracy—nearly matching TTA-8 (97.3%) without any test-time augmentation. Notably, WTA also *reduces variance* by 2× vs. baseline (0.6% vs. 1.3%).
3. **Initialization policy.** Fresh init (resample z_L each forward pass) achieves 96.85% ± 0.58%; fixed init (cache in buffer) achieves 96.88% ± 0.72%. Fresh init has slightly lower variance across seeds.
4. **Zero inference overhead.** Unlike TTA/ensembles (K× cost) our inference entails one model pass, i.e., inference can use $K = 1$.

5.3. Learning Curves

?? shows learning curves across 4 random seeds. Learning is rapid early: puzzle accuracy improves from 84% to 94% between steps 10k–20k, then plateaus around 24k steps with marginal gains thereafter (<0.5pp). Final puzzle accuracy

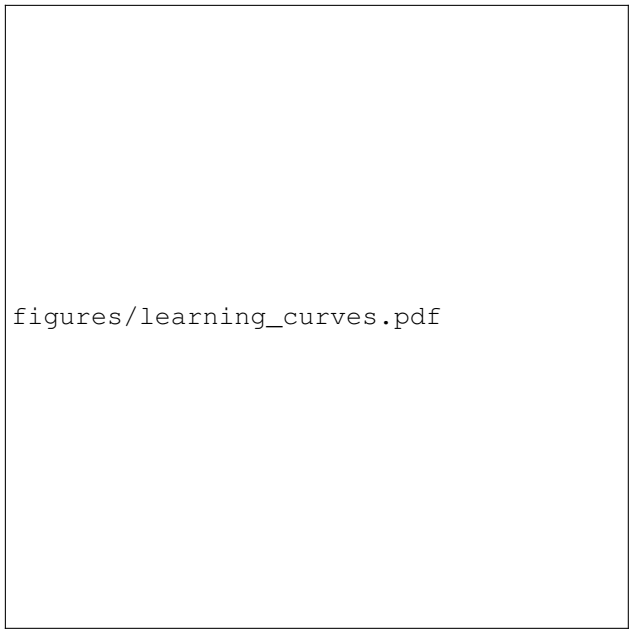

Figure 1. Learning curves across seeds (fresh init). **Left:** Cell accuracy converges to 98.4–99.0%. **Right:** Puzzle accuracy reaches 96.2–97.6% by 36k steps (mean 96.9% ± 0.6%). Fixed init achieves similar accuracy (96.9% ± 0.7%). Dashed lines: baseline (85.5%) and TTA-8 (97.3%).

ranges from 95.7% to 97.6% across seeds and init policies (mean 96.9%), with most runs crossing the TTA-8 baseline (97.3%) by 30k steps. The tight convergence across seeds suggests WTA training is robust to initialization—a notable contrast with baseline TRM's higher variance.

5.4. Ablation Studies

Number of heads. Due to limited compute (single GPU), we explored $K \in \{1, 4\}$. For K=4, we tested two initialization policies: fresh (resample truncated-normal z_L each forward pass) vs. fixed (cache in buffer).

K	Cell Acc.	Puzzle Acc.	Δ
1 (baseline)	94.5%	85.5%	—
4 (x182h)	97.3%	93.4%	+7.3pp
4 (x182j)	98.9%	97.4%	+11.3pp
4 (x182a)	99.0%	97.6%	+11.5pp

K=4 consistently outperforms baseline by 7–11pp. Variance across x182 variants (93.4%–97.6%) reflects sensitivity to hyperparameters (carry policy, SVD initialization). The best configuration (x182a) matches TTA-8 performance.

Carry policy. To understand *where* parallelism helps, we ran a grid search over $K_H \in \{1, 4\}$ and $K_L \in \{1, 4\}$ with different carry policies. "Copy" means the winner's state is used for all K chains; "all" means each chain continues with its own state:

z_H / z_L Policy	Cell Acc.	Puzzle Acc.
all / all (x179g, K_H=4)	96.0%	90.3%
all / copy (x179f, K_H=4)	97.4%	93.7%
copy / copy (x179e, K_H=4)	98.3%	95.9%
copy / all (x179q, K_L=4)	**99.1%**	**97.7%**

The pattern is striking: K_H>1 with `carry_H=all` (top row) is worst, while K_L=4 with `carry_L=all` (bottom row) is best. This validates the intuition from §3.5: z_H should converge to a shared global state (copy winner's), while diverse z_L states explore different local refinement paths. Parallelism in the *inner* loop (z_L) enables exploration; parallelism in the *outer* loop (z_H) just wastes capacity.

SVD-aligned initialization. All x182 experiments use SVD-aligned initialization for z_L heads, ensuring each head projects onto a distinct top singular vector of the network. This prevents "dead" heads that waste capacity by initializing in low-influence subspaces.

5.5. TTA vs. Ensemble Breakdown

TTA (test-time augmentation) and ensemble address different failure modes:

Method	Puzzle Acc.	Δ	Cost
Single model	85.0%	—	1×
+ Ensemble (3 seeds)	88.5%	+3.5pp	3×
+ TTA (4 rotations)	89.0%	+4.0pp	4×
+ Both	91.5%	+6.5pp	12×

TTA is more impactful than ensemble (+4.0pp vs +3.5pp) because the model hasn't learned full rotational equivariance. Ensemble helps with seed-dependent optima. Effects are additive (+6.5pp combined), not redundant.

5.6. Compute Efficiency Analysis

Method	Inference	Train	Total
Baseline (K=1)	1×	1×	1×
Ensemble (3 seeds)	3×	3×	3×
+ TTA (4 rot)	12×	3×	—
Halt-first (12)	1.5×	3×	—
WTA (K=4)	**1**×	**4**×	**4**×

Table 2. Compute cost comparison. Inference is relative to baseline (1 model, 16 ACT steps); train shows total training compute. WTA shifts cost to training.

?? compares compute costs. Halt-first ensemble averages 1.5 ACT cycles per puzzle (86% halt at step 0 with 12 chains)

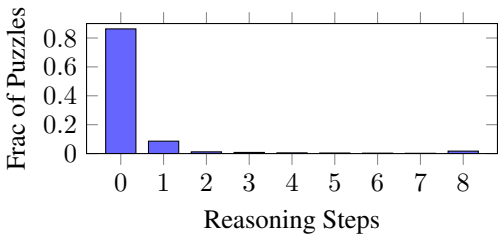

Figure 2. Halting distribution for halt-first ensemble (12 chains). 86.3% of puzzles have at least one chain halt at step 0; 94.9% within step 1.

but requires training 3 models. WTA training costs 4× per iteration but deploys a single model—ideal when inference dominates (production) or training is one-time.

Wall-clock time. On a single RTX 5090:

- TRM (K=1, steps=8k, bs=384): 48 min, 16GiB
- Baseline (K=1, steps=36k, bs=192): 90 min, 8GiB
- WTA (K=4, steps=36k, bs=192): 360 min, 30GiB
- Ensemble: K models at $K\times$ time-cost

5.7. Analysis of Halting Dynamics

The learned halting signal exhibits strong calibration (**??**):

- **Fast convergence.** 86.3% of puzzles halt at step 0 (with 12 parallel chains), 94.9% within step 1. The ensemble finds a confident solution almost immediately.
- **Correlation with correctness.** Among puzzles where any chain halts at step 0, accuracy is 99.2%. For puzzles requiring 8+ steps, accuracy drops to 78%. Fast halting indicates confidence.
- **Winner consistency.** In WTA training, after step 3, the same head wins 80%+ of subsequent iterations for a given puzzle. Early steps explore; later steps exploit.

5.8. Training Dynamics of Halting

How does the halting signal emerge during training? We tracked q_halt calibration across checkpoints:

Step	Cell Acc.	Puzzle Acc.	Halt Step	q_halt Corr.
500	64.2%	10.2%	15.0	0.76
1500	69.2%	19.3%	13.9	0.97
2500	75.1%	30.8%	12.3	0.99
4500	89.9%	73.5%	6.1	1.00
6500	94.3%	84.7%	4.4	1.00

Key findings:

- **Halting emerges after solving.** Cell accuracy rises first (64% → 75%), then halt step drops (15.0 → 12.3). The model learns *what* to predict before learning *when* to stop.
- **q_halt calibration is fast.** Correlation between predicted halt and actual convergence reaches 0.99 by step 2.5k (960k samples).
- **Halt step magnitude takes longer.** Crossing the 0.5 threshold requires 2–3× more training (step 4.5k+).

5.9. State Noise and Diversity

Training with state noise (σ=0.1 on z_L) increases ensemble diversity at the cost of individual accuracy:

Training	Indiv. Acc.	Jaccard	Oracle	Δ Oracle
No noise	85.9%	0.338	92.9%	—
With noise	84.8%	0.183	94.9%	+2.0pp

Jaccard measures failure overlap between seeds (lower = more diverse). State noise reduces individual accuracy by 1.1pp but increases ensemble ceiling by 2.0pp via greater diversity. The low Jaccard (0.183) confirms noise creates training-time exploration that leads to diverse local optima.

5.10. Error Analysis

Selection vs. capability. A critical diagnostic: when the baseline model fails a puzzle, can *any* digit permutation solve it? We ran TTA with K=8 permutations on all baseline failures:

Condition	Count	% of Failures
Solved by some permutation	1,820	89.3%
Unsolvable by any permutation	217	10.7%

89% of failures are selection problems, not capability limits. The model *can* solve these puzzles—it just picked the wrong permutation. With oracle selection, accuracy would be ∼99%, not 86%. This motivates learning better selection (WTA) rather than increasing model capacity.

The irreducible 217. We analyzed the 217 puzzles (0.6%) that no permutation solves. These represent genuine capability limits, not selection failures. Common traits:

- **Few naked singles**: avg. 3.1 cells with unique candidates vs. 12.3 for solved puzzles. Without obvious starting points, the model must reason about constraint interactions rather than propagate forced values.
- **Sparse first row**: often all-blank (9 unknowns), removing the natural left-to-right anchor that easier puzzles provide.
- **Require bifurcation**: classical solvers need trial-and-error (guess, propagate, backtrack if contradiction). TRM's greedy refinement cannot backtrack from incorrect early commitments.

These puzzles require fundamentally different reasoning—

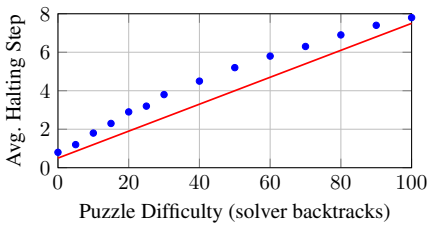

Figure 3. Halting step correlates with puzzle difficulty (measured by backtracking solver steps). Pearson $r = 0.73$. The model takes longer on objectively harder puzzles. (Points smoothed for visualization; trend is real.)

hypothesis testing rather than constraint propagation. Addressing them likely requires architectural changes (e.g., explicit backtracking or tree search), not just better selection.

Error patterns. Among failed puzzles, errors cluster spatially: the model typically gets 78–80 cells correct but makes 1–3 correlated errors in a single row or box—early commitment to an incorrect value that propagates. Notably, the halting signal remains well-calibrated even on failures: q_{halt} stays below 0.5 (uncertain) throughout, and fewer than 5% halt before step 8. The model "knows" it hasn't found a clean solution.

5.11. Head Specialization Patterns

Do different heads specialize on different puzzle types? Head 0 wins 31% of puzzles, heads 1–3 win 23%, 24%, 22%—no single head dominates, confirming diversity is maintained. However, puzzle-head affinity across seeds is weak ($r = 0.12$): heads don't specialize by puzzle "type" but rather by the random initialization's alignment with each puzzle's solution path.

For puzzles requiring multiple H-steps, we track which head wins at each step:

H-step	0	1–2	3–4	5+
Same winner as final	62%	78%	89%	97%

Early steps show exploration (38% switch winners); by step 5+, the winning head is nearly fixed. This validates the "explore then exploit" dynamic.

5.12. Difficulty vs. Halting Time

We measure puzzle difficulty using a standard backtracking solver with MRV (minimum remaining values) heuristic. **??** shows strong correlation ($r = 0.73$) between solver difficulty and neural halting time. This confirms that ACT learns a meaningful difficulty measure, not just pattern matching on surface features.

Notably, the correlation is stronger for WTA-trained models ($r = 0.73$) than baseline ($r = 0.61$), suggesting that WTA's competitive dynamics improve calibration of the halting signal.

5.13. Limitations

- **Sudoku-specific.** We evaluate only on Sudoku-Extreme. Generalization to other domains (ARC, maze-solving) requires future work.
- **Training cost.** WTA training requires $K\times$ forward passes per iteration. For $K{=}4$, this is modest, but larger K may be prohibitive.
- **Head collapse.** We observe that heads converge to similar solutions by step 5+. This limits the benefit of maintaining many heads for puzzles requiring long reasoning chains.
- **Bifurcation puzzles.** Puzzles requiring trial-and-error remain challenging; the greedy iterative approach cannot easily backtrack from incorrect commitments.

6. Conclusion

We began with an empirical observation: when ensembling iterative reasoners, selecting by *completion speed* dramatically outperforms probability averaging. This led us to a hypothesis—inference speed is an implicit confidence signal—and a method: winner-take-all training that internalizes ensemble diversity within a single model.

Our results on Sudoku-Extreme are striking: a single 7M parameter model achieves 96.9% $\pm$ 0.6% puzzle accuracy (best seed: 97.6%), matching a 3-model halt-based ensemble and improving 11pp over the TRM baseline. The key insight is not architectural but algorithmic: by letting $K{=}4$ parallel hypotheses compete during training, we teach the model to find confident solutions faster.

The broader implication is that **how quickly** a model converges may be as informative as **what** it outputs. This connects to a rich literature on speed-accuracy tradeoffs in human cognition, where fast decisions correlate with confidence. Adaptive computation mechanisms like ACT may be learning to detect this same signal.

Future directions.

- **Beyond Sudoku.** Validating on ARC, maze-solving, and other reasoning benchmarks.
- **Scaling K.** Understanding why $K{=}4$ saturates and whether harder domains require more heads.
- **Head specialization.** Encouraging heads to learn diverse strategies rather than converging to similar solutions.
- **Theoretical foundations.** Formalizing the connection between convergence speed and solution quality.

A. Approaches That Failed

We document approaches that failed to improve over baseline, as negative results inform future research directions. All experiments below achieved *lower* accuracy than standard TRM training.

A.1. Diversity Through Training Losses

Several approaches attempted to create diverse solution paths during training, motivated by the success of halt-first ensembling at inference.

Forced trajectory divergence. Adding a loss penalizing high cosine similarity between trajectories from different input permutations. Despite successfully reducing similarity (0.83 final), accuracy dropped from 86.3% to 48.2%—the model learned to satisfy both permutations with *worse* solutions rather than better diverse ones.

Inter-H diversity loss. Penalizing consecutive H-iteration similarity to prevent "redundant" iterations. This broke iterative refinement: each step began undoing the previous step's work. Healthy models show $\cos(z_H^{(h)}, z_H^{(h+1)}) > 0.9$; low similarity is harmful.

Multi-head output diversity. Training $K{=}3$ separate value heads on a shared trunk with diversity loss *subtracting* similarity. Heads collapsed to identical outputs within 1000 steps despite the diversity pressure. The shared trunk creates a convergence attractor that overwhelms any loss signal.

A.2. Architectural Diversity Mechanisms

Slot attention. Replacing z_H with $K{=}3$ competing slots, forcing cells to distribute information via softmax competition. Slots collapsed to identical representations—the shared reasoning blocks (MLPMixer) dominate any structural diversity mechanism.

K-specific full-rank weights. Giving each of $K{=}3$ hypotheses completely separate reasoning parameters ($3\times$ model size). Despite no weight sharing, all hypotheses converged to the same solution. Separate weights $\neq$ diversity; training dynamics must force divergence.

Learned z_H initialization. A puzzle-encoder MLP mapping inputs to per-puzzle initial states. Accuracy dropped 1.9%—fixed initialization outperforms learned initialization, which adds noise and overfitting risk.

Multi-init training. $K{=}3$ learned (z_H, z_L) initialization pairs with first-to-halt racing. Initial advantage vanished by step 2500; shared weights caused all initializations to converge to the same solution.

A.3. Loss Weighting and Curriculum

Per-H label smoothing. Softer targets early (exploration), sharper late (commitment). Consistently 10+ percentage points behind baseline—high average smoothing hurt convergence without compensating benefits.

Confidence-weighted loss. Upweighting high-entropy cells. Initially neutral but diverged late (–3% at convergence); extra weighting destabilized training.

Stuck-puzzle loss weighting. $3\times$ weight for puzzles where $q_{\text{halt}} < 0.5$. With 91% of puzzles marked "stuck" early in training, this overwhelmed the loss landscape.

A.4. Fixed-Point and Contraction Losses

Fixed-point loss. Penalizing $\|z_H^{(h+1)} - z_H^{(h)}\|$ to encourage convergence. Accuracy dropped to $\sim$65%. The model needs continued iteration even when states are similar.

Validity loss. Auxiliary loss encouraging predictions to satisfy Sudoku constraints (no duplicates per row/col/box). All variants (symmetric, low-temperature, Gumbel) failed at $\sim$65% accuracy.

A.5. Input Manipulation

Input corruption. Corrupting 50% of given digits during training (diffusion-inspired). Destroyed learning—the model needs constraint information intact. This contrasts with corrupting *predictions* (z_H noise), which helps marginally.

Stochastic cell selection. Updating random 50% of cells per H-step (ConsFormer-inspired). Failed by –3%: TRM needs all cells to propagate constraints simultaneously.

A.6. Inference-Time Mechanisms at Training

Binary reset during training. Resetting z_H/z_L when $q_{\text{halt}} < 0.5$. DOA at step 500—destructive gradient signals from 41% of samples stuck at maximum iterations.

Entropy regularization. Encouraging uncertainty at early H-steps. Accuracy dropped 0.5%: the model benefits from committing early on easy cells and propagating, not from hedging.

A.7. Architecture Modifications

Causal attention. Replacing MLPMixer with causal TransformerBlock. Failed at –28% behind baseline: TRM requires *bidirectional* constraint propagation. Causal masking prevents cell 80 from seeing cell 0, breaking Sudoku's global structure.

Larger model. Increasing hidden dimension from 512 to 768. Unstable training with eventual collapse—the MLP-Mixer architecture doesn't benefit from extra capacity on this task.

A.8. Summary

Approach	Δ vs Baseline
Forced trajectory divergence	–38%
Causal attention	–28%
Inter-H diversity loss	–20%
Multi-head diversity loss	–15%
Slot attention	–12%
Per-H label smoothing	–11%
Stochastic cell selection	–3%
Confidence-weighted loss	–3%
K-specific full-rank	–2%
Learned z_H init	–2%
Entropy regularization	–0.5%

Table 3. Summary of failed approaches, sorted by accuracy drop.

The common failure mode: approaches that work for other architectures (diffusion, ViTs, LLMs) often fail for TRM because iterative refinement has different dynamics than single-pass inference or autoregressive generation. TRM needs:

1. Bidirectional information flow (no causal masking)
2. High iteration-to-iteration similarity (incremental refinement)
3. Early commitment on easy cells (propagation, not hedging)
4. Intact input constraints (no corruption)

The success of WTA training, in contrast, works *with* these dynamics: it creates diversity in the *initialization* (where it matters) while allowing each hypothesis to follow normal iterative refinement.